MONASH University

MONASH BUSINESS SCHOOL

ETC4500/ETC5450 Advanced R programming

Week 4: R programming with LLMs

Outline

1 Terminology

2 Positron Assistant

3 Other tools

Outline

1 Terminology

2 Positron Assistant

3 Other tools

Terminology

- **Provider**: Service that hosts LLM and provides API for interacting with it (e.g., Anthropic)
- **Model**: Specific version of LLM (e.g., Claude-Sonnet-4.6)
- **Prompt**: Input you give LLM to generate a response.
- **System prompt**: Special type of prompt that sets behaviour of LLM.
- **Tokens**: Units of text that the LLM processes (words, parts of words, punctuation, etc.).
- **Context**: Information available to LLM when generating a response
- **Tool**: Function or API that LLM can call to perform specific tasks

Outline

1 Terminology

2 Positron Assistant

3 Other tools

Set up Positron

GitHub Copilot

- Make sure you are logged into your GitHub account.
- GitHub Copilot settings: `https://github.com/settings/copilot/features`

Anthropic Claude

- Set up Claude as a model provider using an Anthropic API key: `https://platform.claude.com/`
- `Ctrl-Shift-P`: “Positron Assistant: Configure Language Model Providers”

Code completion

GitHub Copilot provides code completion suggestions as you type.

You can accept suggestions by pressing `Tab` or `Enter`

Snooze it by clicking the icon at bottom right of the editor, or by pressing `Ctrl+Shift+P` and selecting “GitHub Copilot: Toggle”

Try typing a function name or a comment describing what you want to do, and see what suggestions Copilot gives you.

Default chat agents

- **Ask**: Provides answers and code suggestions without automatic execution
- **Edit**: Suggests code changes for you to apply to files
- **Agent**: Autonomously executes work including running code and modifying files
- **Plan**: Creates detailed step-by-step plans before execution

You will need a model. Default is GPT-5 mini (slower and less capable than a paid model).

Pick model

- GitHub Copilot: Code completion and chat agents
- Claude: Chat agents with more detailed reasoning and planning capabilities

Chat agent: Ask

- How do I plot a histogram of the `value` variable in my dataset `my_data`?
- Why am I getting a syntax error in my R script?
- Can you explain what this function does?

Attach context

- Default context: active editor file, and active interpreter session.
- Add any other context you think will help.
- Interpreter session context includes code inputs and outputs in that session, in-memory objects in the Variables pane, the active plot in the Plots pane.

Chat agent: Edit

- Refactor this function to be more efficient.
- Add error handling to this code block.
- Update this script to use the latest version of the `dplyr` package.

Chat agent: Agent

- Create a new R script that loads my dataset, performs some basic data cleaning, and generates a summary report.
- Install the required packages and run this analysis script for me.
- Update all my ggplot graphics to use a color-blind friendly palette.

Chat agent: Plan

- Outline the steps needed to turn this script into an R package.
- Plan how to add tests for this R script
- Create a step-by-step plan to optimize this data processing pipeline.

Git help

- You can ask the agent to generate commit messages for you, or even create commits and push them to GitHub.
- You can ask the agent to review your code and suggest improvements, or even automatically apply those improvements.

Slash commands

- `/fix` - Fixes code
- `/explain` - Explains code
- `/exportQuarto` - Generates a Quarto document from the current chat conversation

Prompting tips

- Be specific about what you want the agent to do.
- Provide relevant context to help the agent understand your request.
- Review the agent's suggestions carefully before accepting them, especially if they involve code changes or execution.
- Use the planning agent for complex tasks that require multiple steps or reasoning.

Exercise

- Ask the `Edit` agent to update the `fizzbuzz()` function to allow numbers other than 5 and 7 to be used.
- Use the `Edit` agent to produce a roxygen header and unit tests for the `fizzbuzz()` function we wrote in week 2 (`http://arp.numbat.space/week4/fizzbuzz.R`).
- Ask the `Plan` agent to outline the steps needed to turn the `fizzbuzz()` function into an R package, including adding documentation and tests.

Outline

1 Terminology

2 Positron Assistant

3 Other tools

Other tools

- Databot: a Positron extension that provides data analysis capabilities using LLMs, based on the same set up as Positron Assistant.
- ellmer package: A package to interact with LLMs from R, including prompt templates and agent frameworks.
- shinychat: Build your own custom chat interface using Shiny, and connect it to an LLM using the ellmer package.

Databot

- Interactive data analysis tool that allows you to ask questions about your data and get answers in the form of code, visualizations, and explanations.
- Uses LLMs to generate code and explanations based on your questions.
- Can write Quarto reports based on your data and analysis.

ellmer package

- Allows you to interact with LLMs programmatically from R.
- Beware of long conversations. The entire conversation history is sent with each request.
- Allows a system prompt to set the behaviour of the agent.
- Allows tools to be registered.
- Provides a framework for building custom agents